DSSP PA 01 Report

20261099 양예진

I. 배경 지식 조사

1. 오디오 포맷

오디오를 저장하고 수정하고 전송할 수 있는 형태로 만들려면 오디오를 인코딩(부호화)하고 디코딩(복호화)하는 일관된 규칙이 필요하다. 그 규칙이 무엇이냐에 따라 포맷이 분류된다.

A. 손실 압축 vs. 무손실 압축

- **Type 1. 원본** : .pcm, .wav, ...
- **Type 2. 무손실 압축(100% 복구 가능)** : .flac, .alac, .ape, wv, ...
- **Type 3. 손실 압축(100% 복구는 불가 but 효율 추구)** : .mp3, .aac, .opus, .wma, ...

Type 1도 원본 아날로그 신호와 완벽히 동일하지는 않으나, 인간의 가청 주파수 대역 밖의 정보 손실은 청각적으로 구분하기 어려우니 이를 raw data로 간주할 수 있다. 압축 과정에서 중요하지 않은 범위를 제거하는 등의 loss가 생기면 손실 있다고 본다.

B. wav = pcm + header

- .wav, .pcm 둘 다 raw representation의 데이터를 담는다. Analog Signal에서 Sampling과 Quantization을 거쳐서 얻은 Digital Signal 정보가 그대로 들어있다.
- .pcm에는 metadata 정보가 없는데 .wav는 헤더에 그게 담겨있다. Sampling Rate, Bit Depth, # of Channel 등의 포맷 설명 정보를 얻을 수 있다.
- 인간 가청 주파수 범위가 $20\mathrm{kHz}$ 근처라서 $f_s > 2f_{max}$에 의거하고, 거기에 여유 대역까지 고려하여 $44.1\mathrm{kHz}$로 샘플링하면 raw 데이터로 적절한 수열이 생성 된다.

C. Metadata?

- sampling rate, bit depth는 각각 샘플링과 퀀타이제이션 영역에서 아날로그 신호를 디지털 신호로 만들 때 중요하게 작동하는 두 가지 변수이다.
- mono / stereo는 재생 시 좌우 채널의 독립성 여부를 결정한다.
- 그 외에도 INFO 청크 등 메타데이터가 달릴 수 있으나, 이번 과제에서 사용한 wavfile 라이브러리에선 이런 정보들은 읽지 않는다고 한다.

2. With or Without Saturation?

A. Saturation이란?

- audio signal을 데이터로 저장하고자 하면 quantizaion 과정에서 필연적으로 '진폭의 상한선'이 생긴다. 그 상한선을 넘게 된다면 어떻게 될까?
- 컴퓨터가 하는 연산 상 bit depth로 인한 상한선/하한선을 넘으면 overflow가 일어나는 게 기본적 작동이다. 그렇게 되면 사람이 듣기에 기분 나쁜 glitch가 생겨버린다.

- 그러므로 overflow가 아니라 clipping이 일어나야 한다. 진폭이 상한선 넘을 경우 넘는 부분을 최대 진폭으로 설정해서 파형의 머리 부분을 깎는 건데, 이걸 위해서 **Saturation Arithmetic** 포화 연산을 해줘야 한다.
- 당연히 파형의 머리가 깎이는 것은 원본과 다른 소리가 들리게 한다. 클리핑을 하면 할 수록 사인파에서 사각파에 가까워진다. 그리고 사각파에 가까워질수록 고주파 배음이 다수 생겨난다.

II. 결과

1. 코드 링크

II-2에 기재된 결과를 얻은 코드와 그 결과로 생성된 .wav 파일들을 깃허브에서 확인할 수 있다.

26-Spring-DSSP/DSSP_PA_1 at main · Yejin-02/26-Spring-DSSP

Contribute to Yejin-02/26-Spring-DSSP development by creating an account on GitHub.

https://github.com/Yejin-02/26-Spring-DSSP/tree/main/DSSP_PA_1

Yejin-02/**26-Spring-DSSP**

1 Contributor 0 Issues 0 Stars 0 Forks

2. 결과 분석 1

A. dssp_pa_1_original.wav

https://file-examples.com/index.php/sample-audio-files/sample-wav-download/ 에서 1MB 파일을 다운로드 받았다.

1. sampling rate: 44100
2. # of samples: 262094 ⇒ duration = $262094/44100$ = 5.943 sec
3. mono or stereo: stereo
4. dtype: `int16`

   ⇒ 각 원소는 $[-32768, 32767]$ 내의 정수이다.
5. 모든 원소를 plot한 결과

   ⇒ 샘플 개수가 너무 많아서 자세하게 확인하기 어렵다.

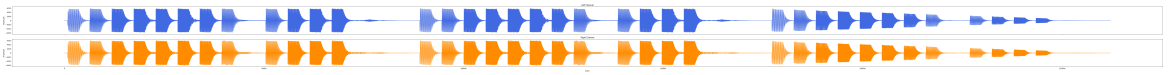


Figure 1. 원본 오디오 신호 L/R 파형 (전체 샘플)

6. 상위 2000개 샘플한 plot한 결과

   ⇒ sine파 형태고, 진폭의 절대값이 6000 근처임을 확인했다.

   ⇒ `max()` 를 이용해 확인해보니 실제로 L에서 5925, R에서 6056이었다.

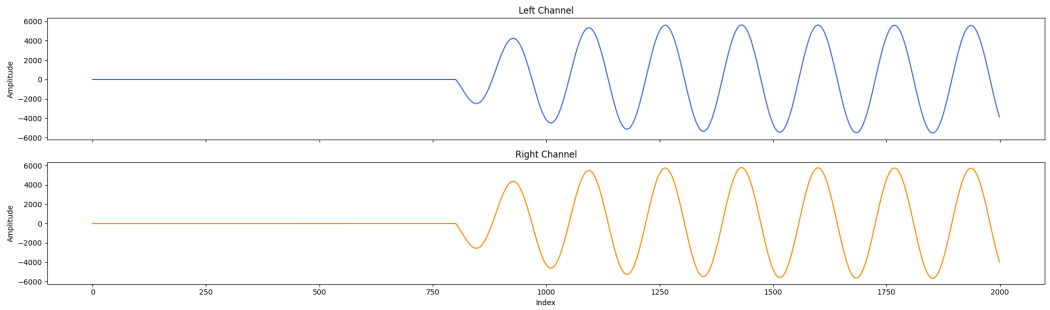


Figure 2. 원본 오디오 신호 L/R 파형 (상위 2000개 샘플)

7. 원본 파일을 재생해보면 부드러운 소리가 들린다. 거의 이상적인 사인파에 가깝다고 생각이 되고, 그래서 순음에 가까운 소리가 나되 어택과 릴리즈로 인해서 삐- 소리처럼 들리지는 않고 자연스러운 악기 연주 소리처럼 들린다.

B. dssp_pa_1_dc_offset.wav

`data + 4096` 수행 결과 헤드룸이 넉넉하여 dc offset만 일어남을 확인했다.

1. `+ 4096` 한 것과 원본 wav를 상위 10000개 샘플에서 plot한 결과

   ⇒ dc offset, 그러니까 진폭 축에서 평행 이동이 무사히 일어날 뿐, 최대 진폭값에 도달할 수 없음을 확인했다.

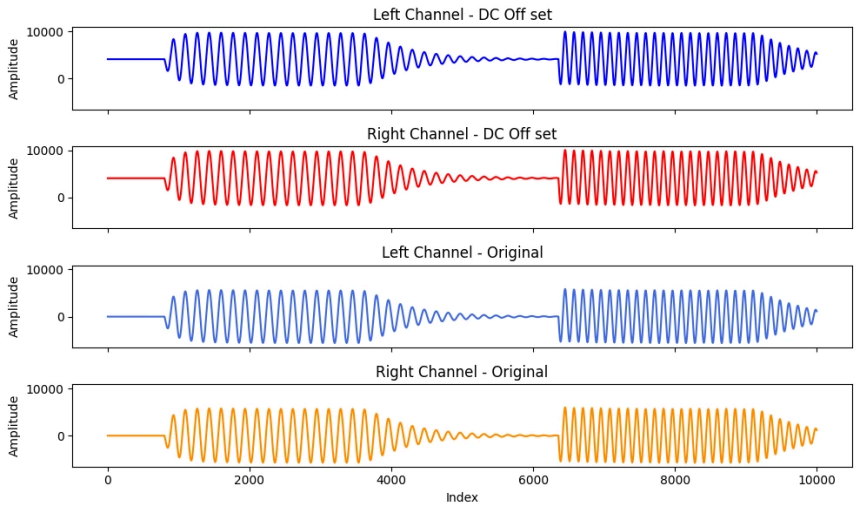


Figure 3. DC offset 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (상위 10000개 샘플)

2. dc offset이 일어난 파일을 재생해보면 큰 볼륨으로 재생했을 때 미묘한 노이즈가 생긴다. 계속해서 들리는 이 노이즈는 아주 낮게 웅웅 울리며 듣는 내내 불편하긴 하지만 귀가 아프진 않다.
3. 이 신호의 맨 끝 부분에서 클릭 노이즈가 발생했다. 이는 dc offset으로 인해 원본에서 맨 끝 부분 페이드 아웃으로 잘 끝나게 처리해둔 효과가 상실됐기 때문으로 예상된다.

C. dssp_pa_1_without_saturation.wav

따라서 `int16`에서 overflow를 연출할 수 있도록 `data * 5` 후 `data + 4096` 수행해보았다.

1. `data * 5 + 4096` 한 것과 원본 wav를 상위 1500개 샘플에서 plot한 결과

   ⇒ 아까와 다르게 진폭 축에서의 평행 이동이 무사히 일어나지 못하고, overflow가 발생함을 확인했다.

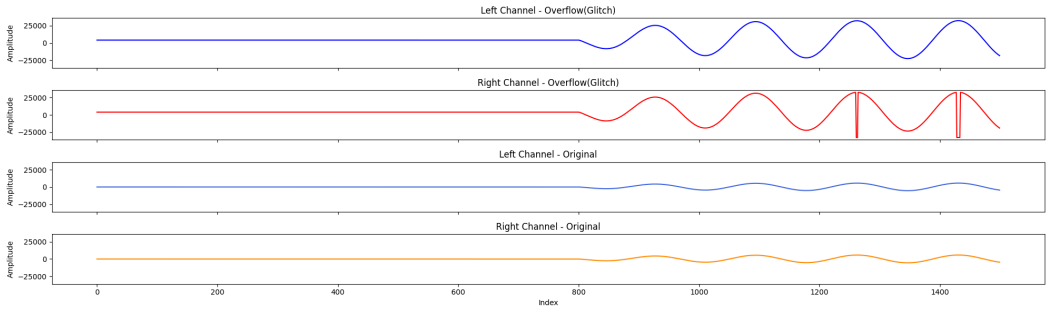


Figure 4. Overflow 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (상위 1500개 샘플)

2. overflow가 발생한 파일을 재생해보면 굉장히 듣기 싫은 형태의 팍 튀는 글리치가 생긴다.
   a. 왼쪽과 오른쪽이 서로 다르게 들리는 stereo 타입의 시그널이고, 위의 그림에서도 R 채널에서만 overflow 생기는 지점이 있는 걸 확인할 수 있듯이 글리치 발생 시각이 왼쪽 오른쪽에서 동일하지 않다.
   b. 하지만 여기까지 인지하기에는 일반적인 사람의 귀로는 어려울 가능성이 높다. 이를 제대로 인지하려면 보다 느리게 재생하는 과정이 필요할 수도 있다.
3. 또한 곱셈 이후에 덧셈 연산이 있기 때문에 여기서도 dc offset이 존재한다. 따라서 신호의 맨 끝 부분에서 클릭 노이즈가 발생한다.

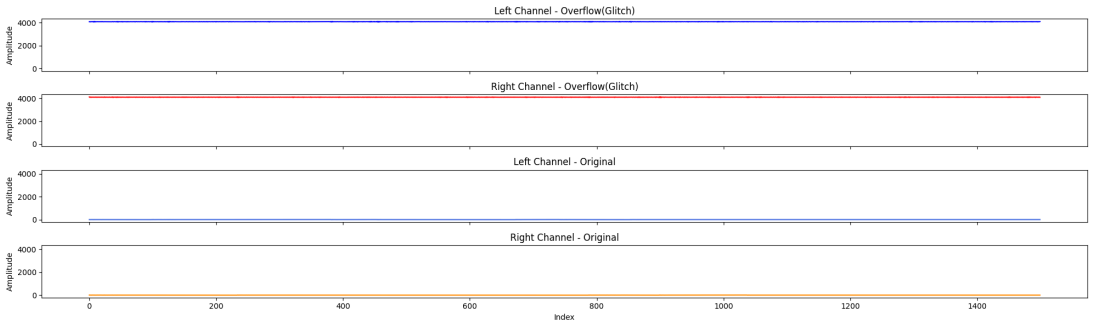


Figure 5. Overflow 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (하위 1500개 샘플)

D. dssp_pa_1_with_saturation.wav

연산 과정에서 clamping이 일어날 수 있도록 하였다. 즉, saturation 연산을 수행했다.

1. `int16`에서 `int32`로 옮겨 헤드룸을 넓힌 후 오버플로우를 방지한 채 `data * 5 + 4096` 연산을 수행하고, 실제 헤드룸으로 옮기기 전 클램핑을 수행한 후 다시 `int16`으로 옮겨온 것과 원본 wav를 상위 1500개 샘플에서 plot한 결과

   ⇒ 오버플로우가 일어나지 않는다.

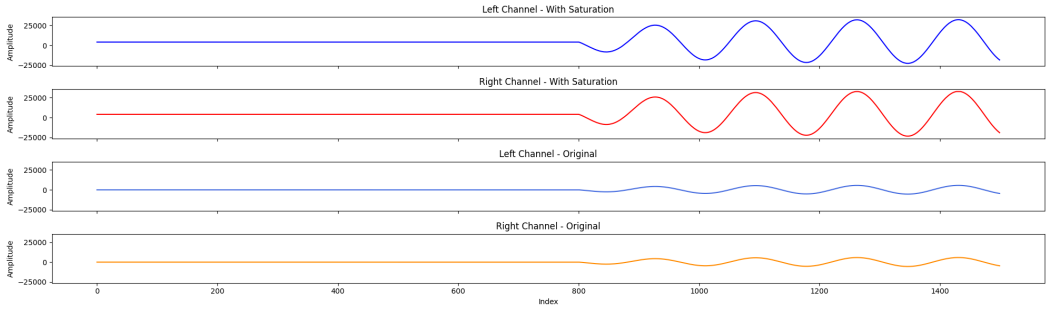


Figure 6. Saturation 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (상위 1500개 샘플)

2. saturation 연산을 수행한 파일을 재생해보면 아까와 같은 귀가 아플 정도의 글리치가 사라진다. 하지만 여전히 그 부분에서 원음과는 다른 느낌이 나타나긴 한다.
3. 여기서도 신호 끝 부분의 클릭 노이즈는 발생한다.

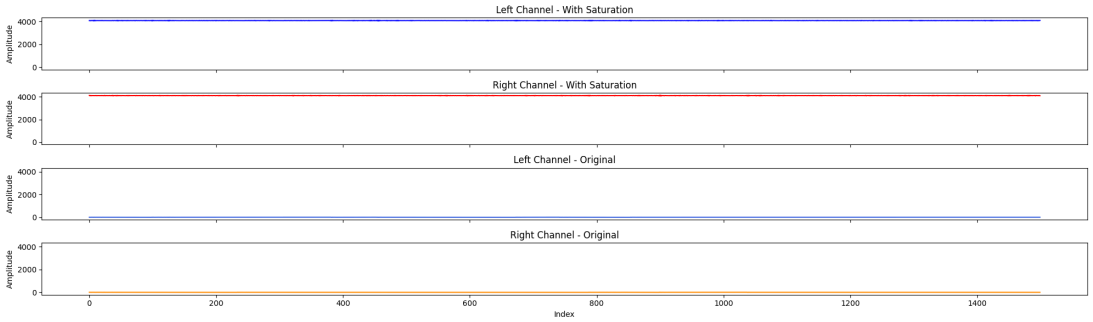


Figure 7. Saturation 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (하위 1500개 샘플)

E. dssp_pa_1_clipping_1.wav

clipping을 많이 하면 사인파가 사각파에 가까워지면서 주파수 대역에서 더 rich하고 buzzy한 사운드가 생기는 효과가 있는데, 위의 예제에선 이를 확인하기 어려워서 추가적인 clipping을 두 가지 수행했다.

1. `data * 50` 을 saturation 연산으로 수행한 것과 원본 wav를 상위 3000개 샘플에서 plot한 결과

   ⇒ 확실히 파형의 머리 부분이 깎여나가는 것이 아까보다 시각적으로 잘 확인된다.

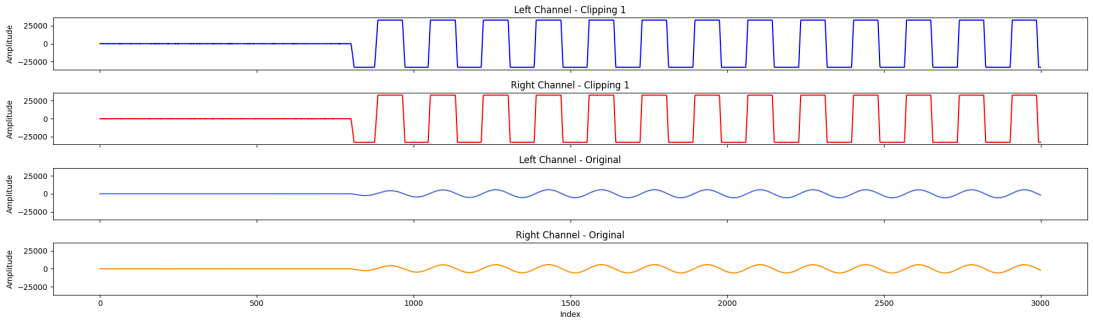


Figure 8. Clipping 1 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (상위 3000개 샘플)

2. clipping 1 파일을 재생해보면 원본 음에서 따뜻하고 부드러운 느낌이 많이 사라졌다. 현실적인 악기 연주보다는 기계적인 사운드에 가깝게 소리가 난다. 거칠다기보단 밝고 경쾌한 무드의 톤이다.

F. dssp_pa_1_clipping_2.wav

더 과감한 clipping도 해보았다.

1. `data * 2000` 을 saturation 연산으로 수행한 것과 원본 wav를 상위 3000개 샘플에서 plot한 결과

   ⇒ 아까보다도 더 사각파에 가까워졌고, 1~800번째 샘플에서 나타나는 노이즈도 극대화됐다.

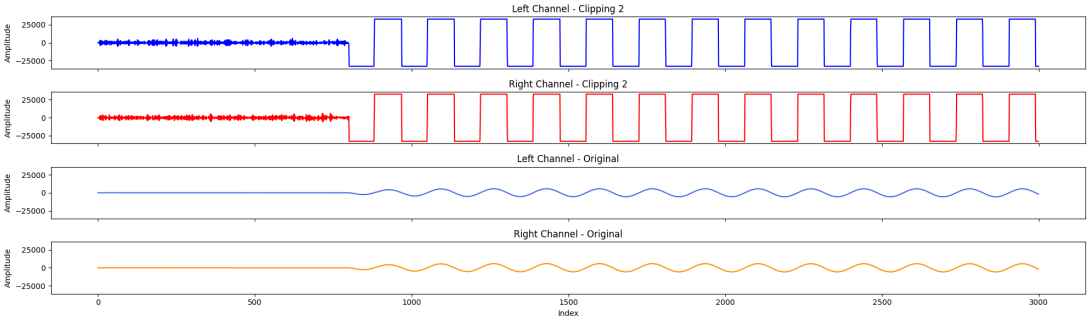


Figure 9. Clipping 2 신호 L/R 파형과 원본 오디오 신호 L/R 파형 비교 (상위 3000개 샘플)

2. clipping 2 파일을 재생해보면 clipping 1보다도 더 거친 소리가 나며 배경 노이즈도 커졌다.
3. Attack/Release 영역에서 원본 - clipping 1 - clipping 2 비교해보았다.
   a. 원본 신호에서 하나의 음을 연주할 때 진폭 포락선을 그려보면 페이드인-페이드아웃 양상이 나타난다.
   b. 진폭을 과하게 키우는 과정에서 그 양상에 변화가 나타난다. 이로 인해 Clipping 된 신호가 더 어택감 있는 소리로 느껴진다.

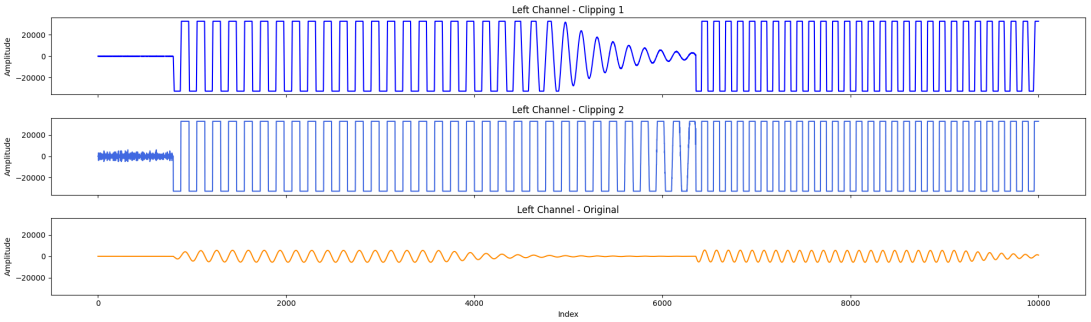
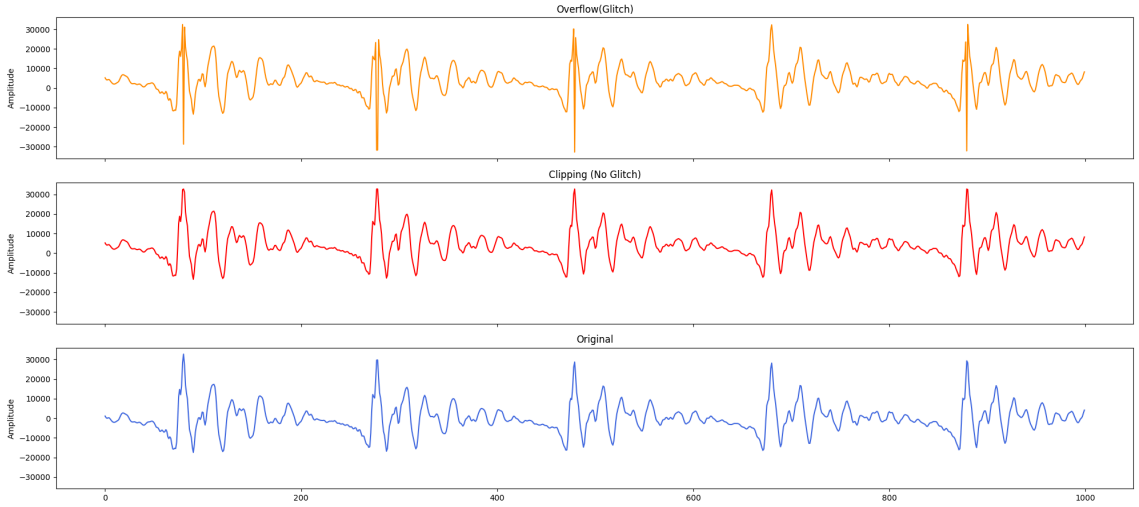
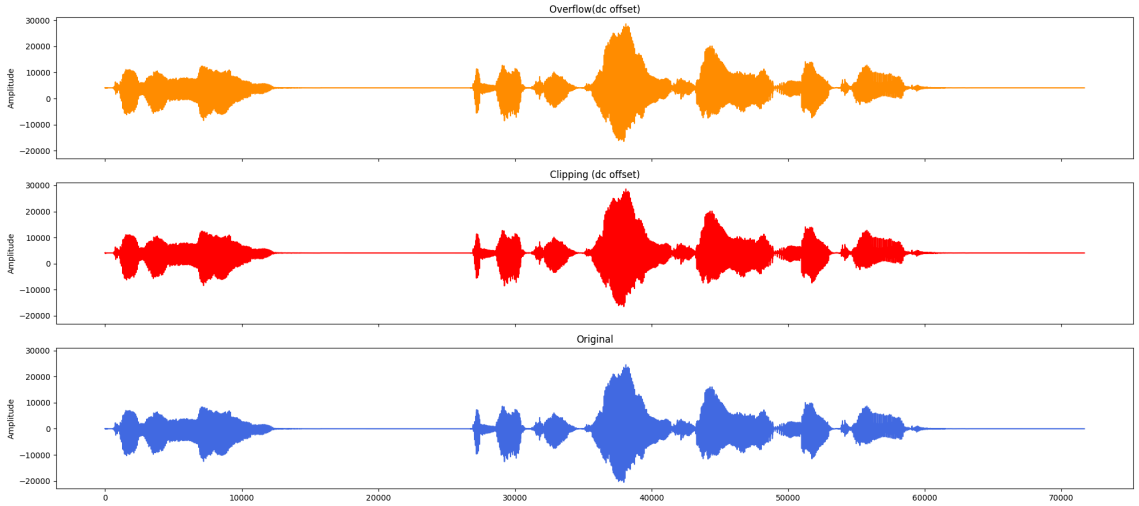


Figure 10. Clipping1 신호 L파형과 Clipping2 신호 L파형, 원본 오디오 신호 L파형 비교 (상위 10000개 샘플)

3. 결과 분석 2





- LMS에 업로드 된 파일 `dssp_pa_1_original_2.wav` 에 대해서도 수행한 결과
  - `without_saturation_data2.wav` 의 경우 18초와 44초에서 글리치 발생,
  - `with_saturation_data2.wav` 에서는 글리치 문제가 해결,
  - 신호 마지막 부분에서 dc offset으로 인한 클릭 노이즈는 둘 다 발생

  함을 확인하였다.